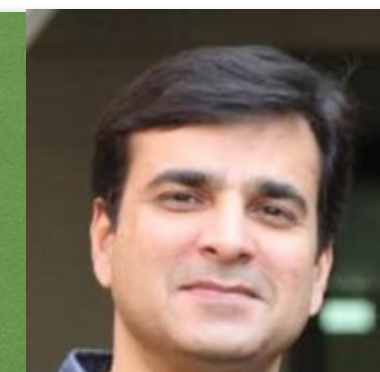


Web Technologies - I



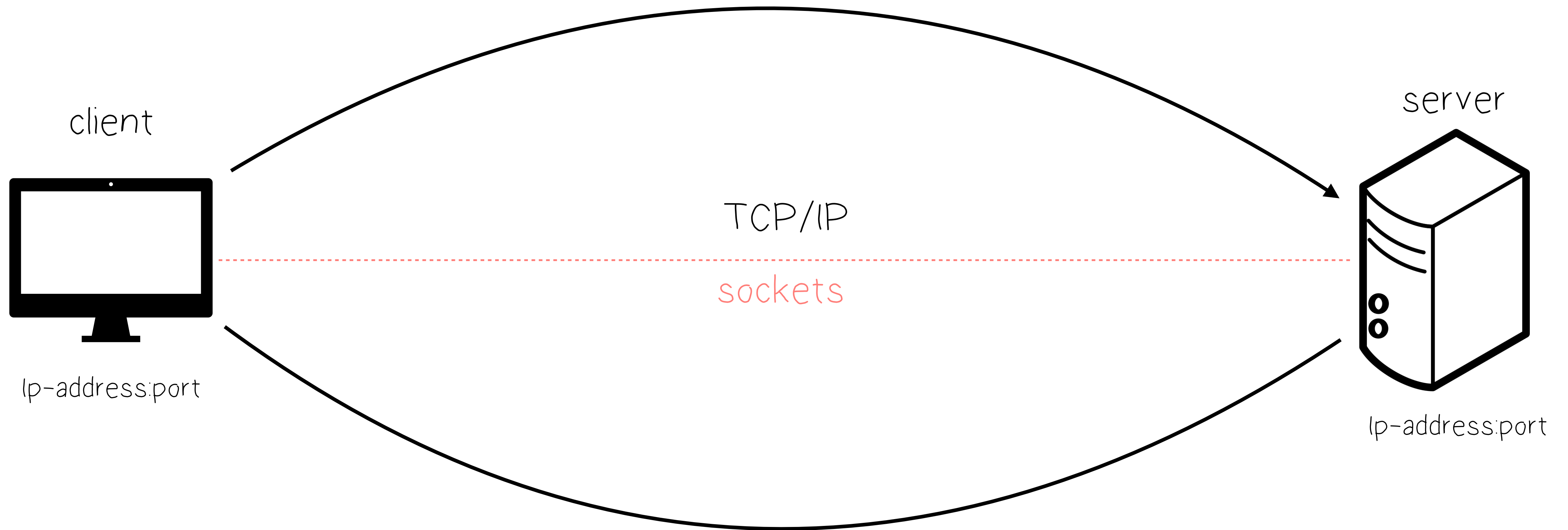
WebTechnologies

Web technologies are the set of tools, protocols, and programming languages used to create, deliver, and interact with content on the **internet**.

They assist in communication to dynamically display and manage content for websites / mobile apps.

Key **client-side** technologies are (HTML, CSS, and JavaScript), whereas Servlets, NextJS or Flask are popular server side frameworks.

High-level View



TCP/IP

(Transmission Control Protocol/Internet Protocol) is the fundamental suite of protocols that allows devices to communicate and exchange data across the internet. Data is transmitted in packets using IP. TCP's job is to make communication reliable, ordered & error free. IP addresses are used for device identification and routing.

IP & Port

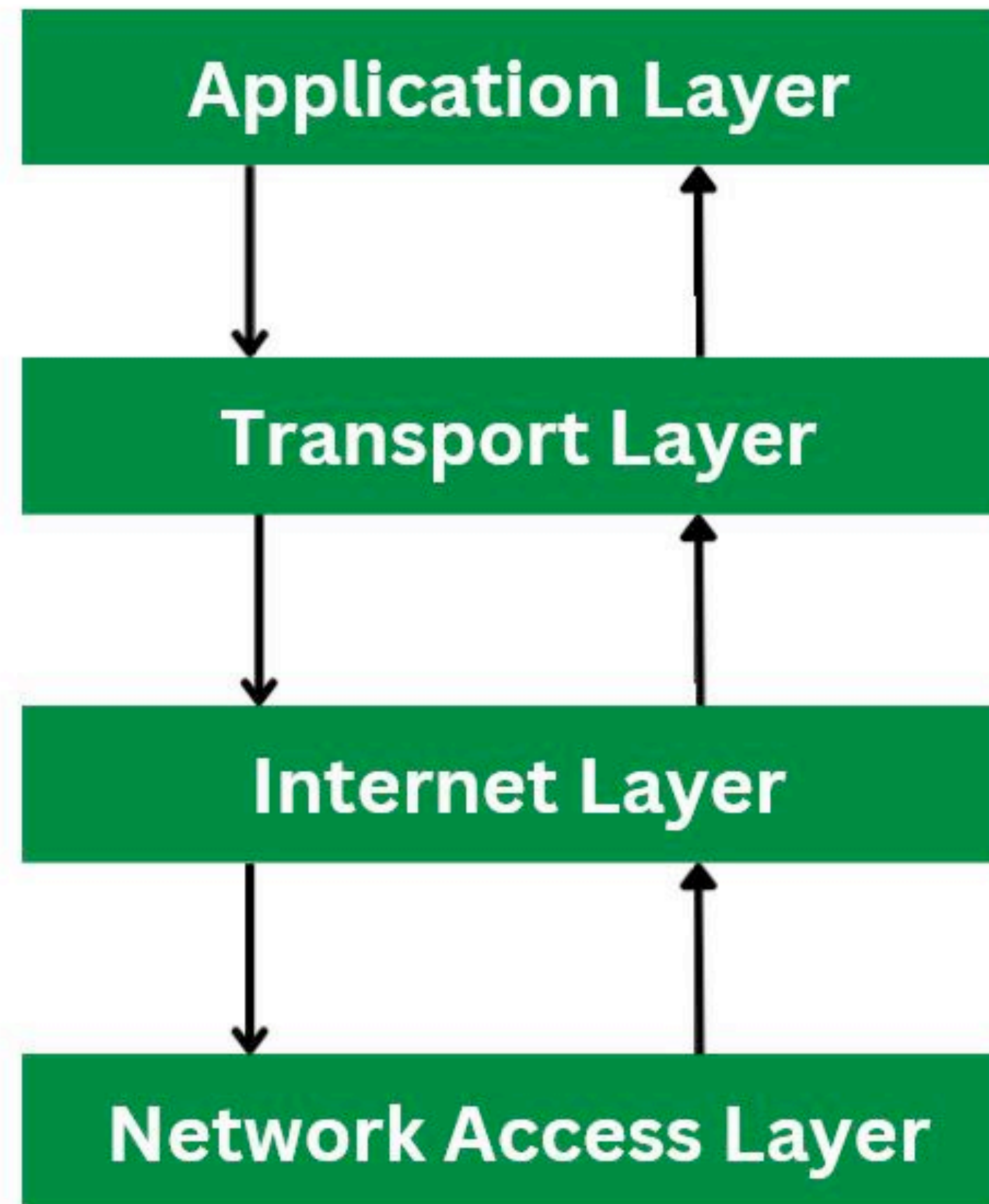
An IP address is a unique numerical identifier for devices on a TCP/IP network.

A computer system can have up to 65,535 virtual network ports, assigned by the operating system and used for running network communication services like web browsing or email.

Well known ports: are ports reserved for known service such as 80 for HTTP, 23 for SMTP etc.

TCP/ IP is a layered
protocol

TCP/IP Layers

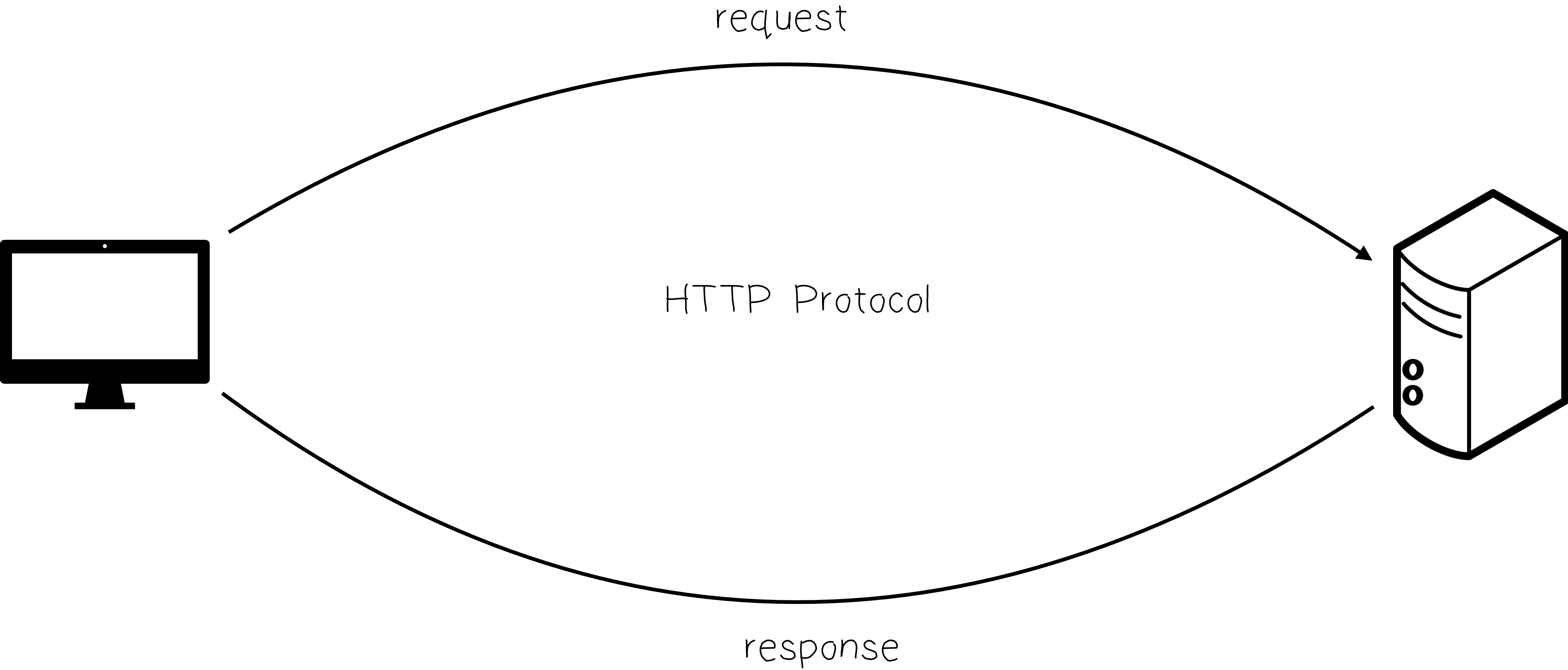


Application Layer

The TCP/IP application layer provides user-facing network services, defining protocols for web browsing (HTTP), email (SMTP, IMAP), file transfers (FTP), and name resolution (DNS).

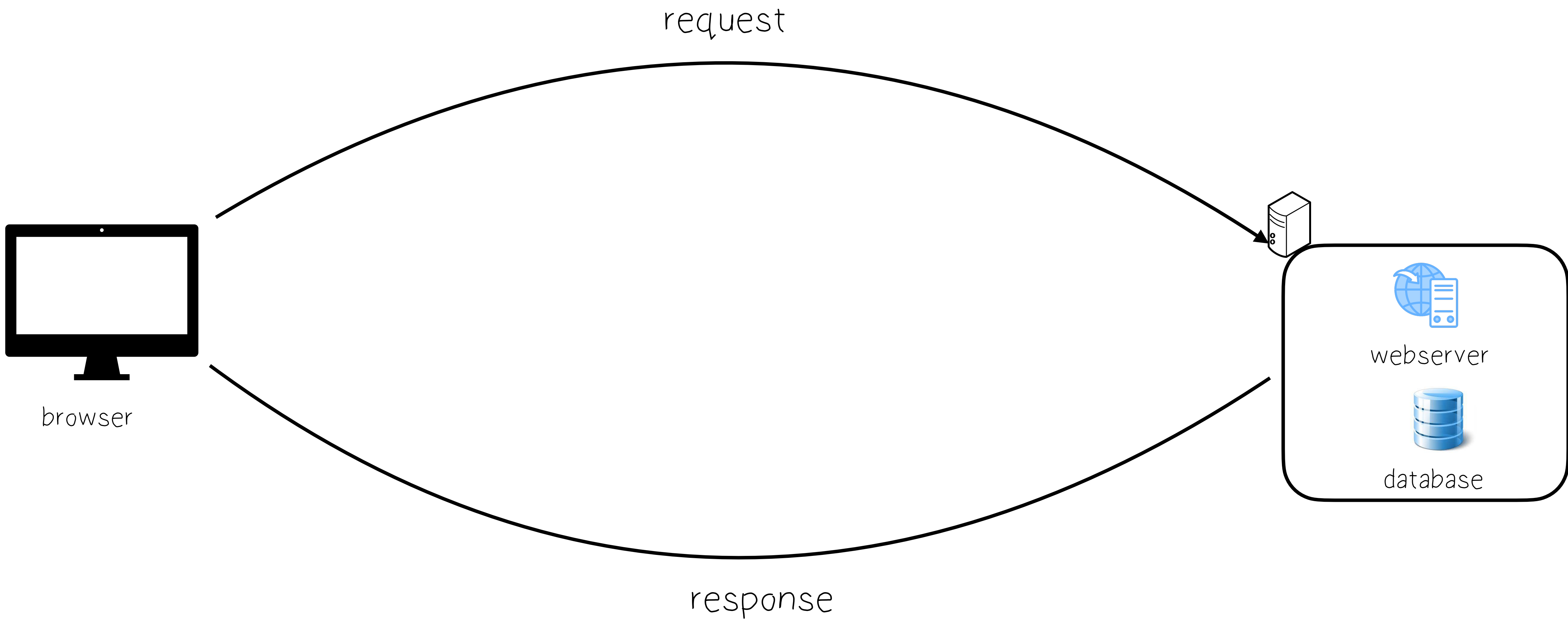
It serves as the interface between software applications and the underlying network.

HTTP View

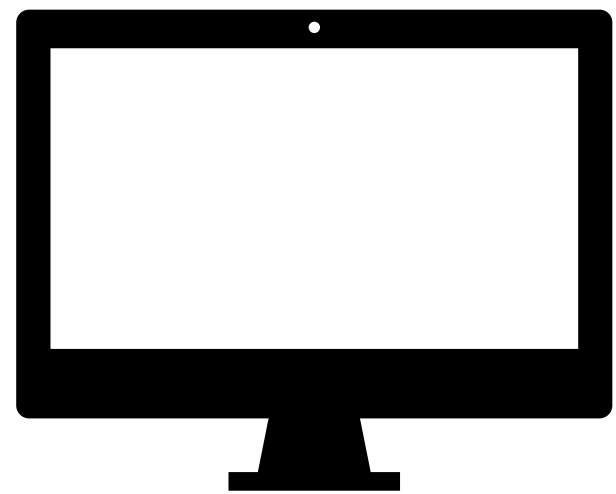


Web Basics

Web or enterprise development involves at least two machines, generally referred to as client & the server. The primary software we use on the client-end machine is the **browser**. **Web server** is the most integral software we need on the server machine.



Developer Views



browser

request

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {
        response.setContentType("text/html");

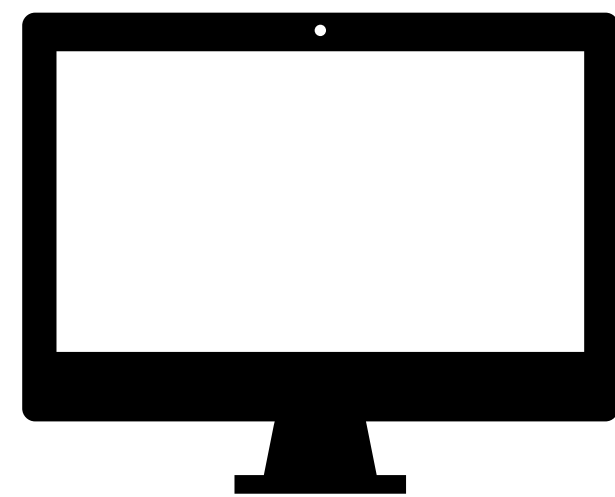
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("</body></html>");
        out.println("<h1>" + message + "</h1>");

    }
}
```



webserver

response



browser

request using URL

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("</body></html>");
        out.println("<h1>" + message + "</h1>");

    }
}
```



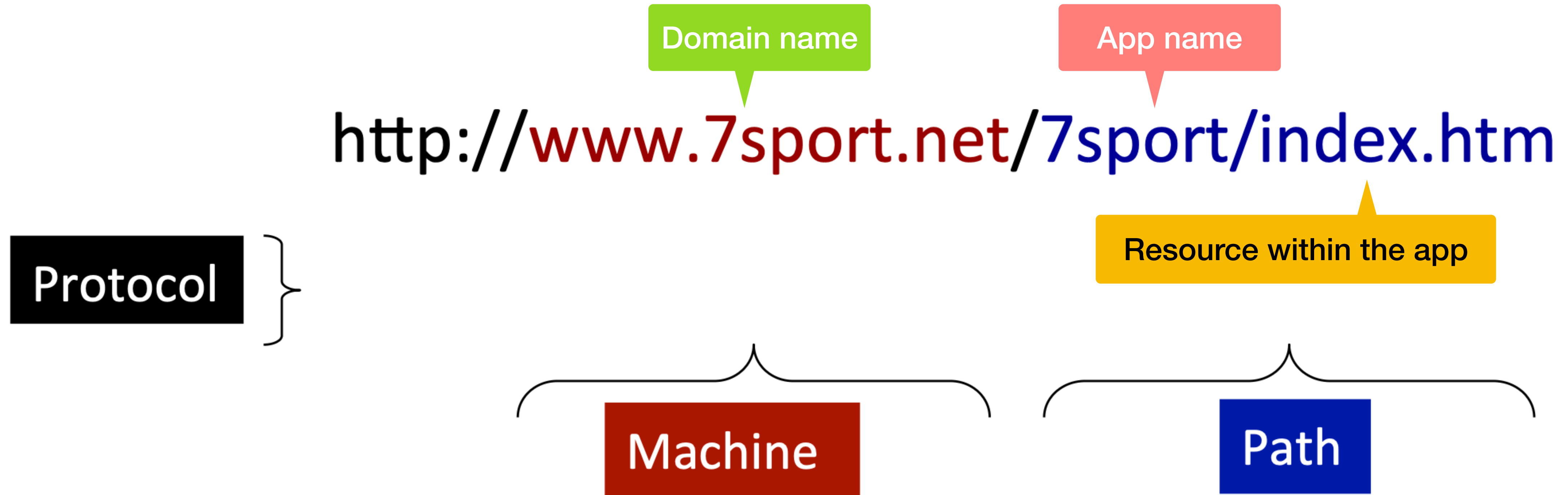
response contains html

What is a URL?

URL Intro.

A web address that identifies the location of a resource, such as a webpage or file, on the web-server.

It's like Mailing address

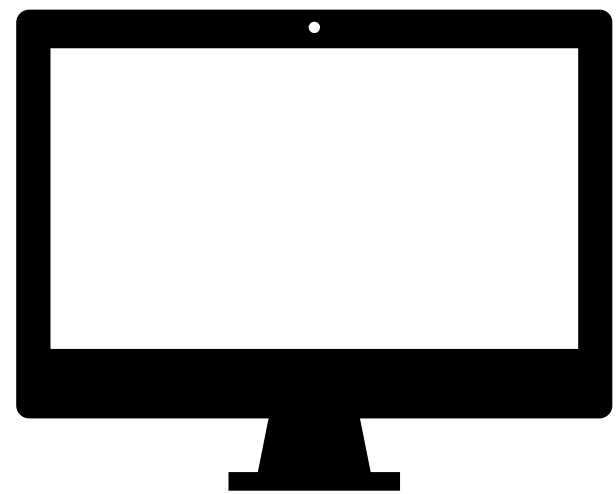


Ways to resolve a machine

Resolve

Using the below options:

- a) domain name (resolve by your ISP's DNS service)
- b) localhost (local domain name defined inside the host file)
- c) 127.0.0.1 (a loopback address)



browser

request

App resides within
Tomcat Server

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {
        response.setContentType("text/html");

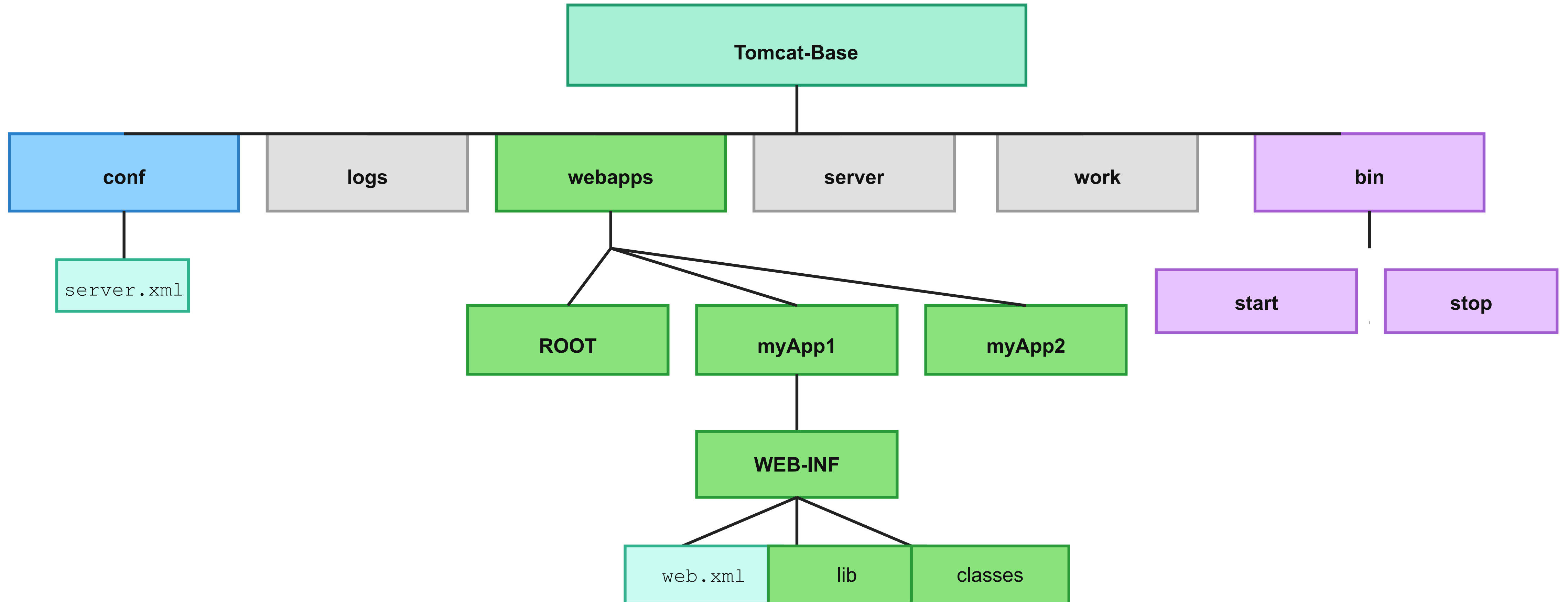
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("</body></html>");
        out.println("<h1>" + message + "</h1>");
    }
}
```



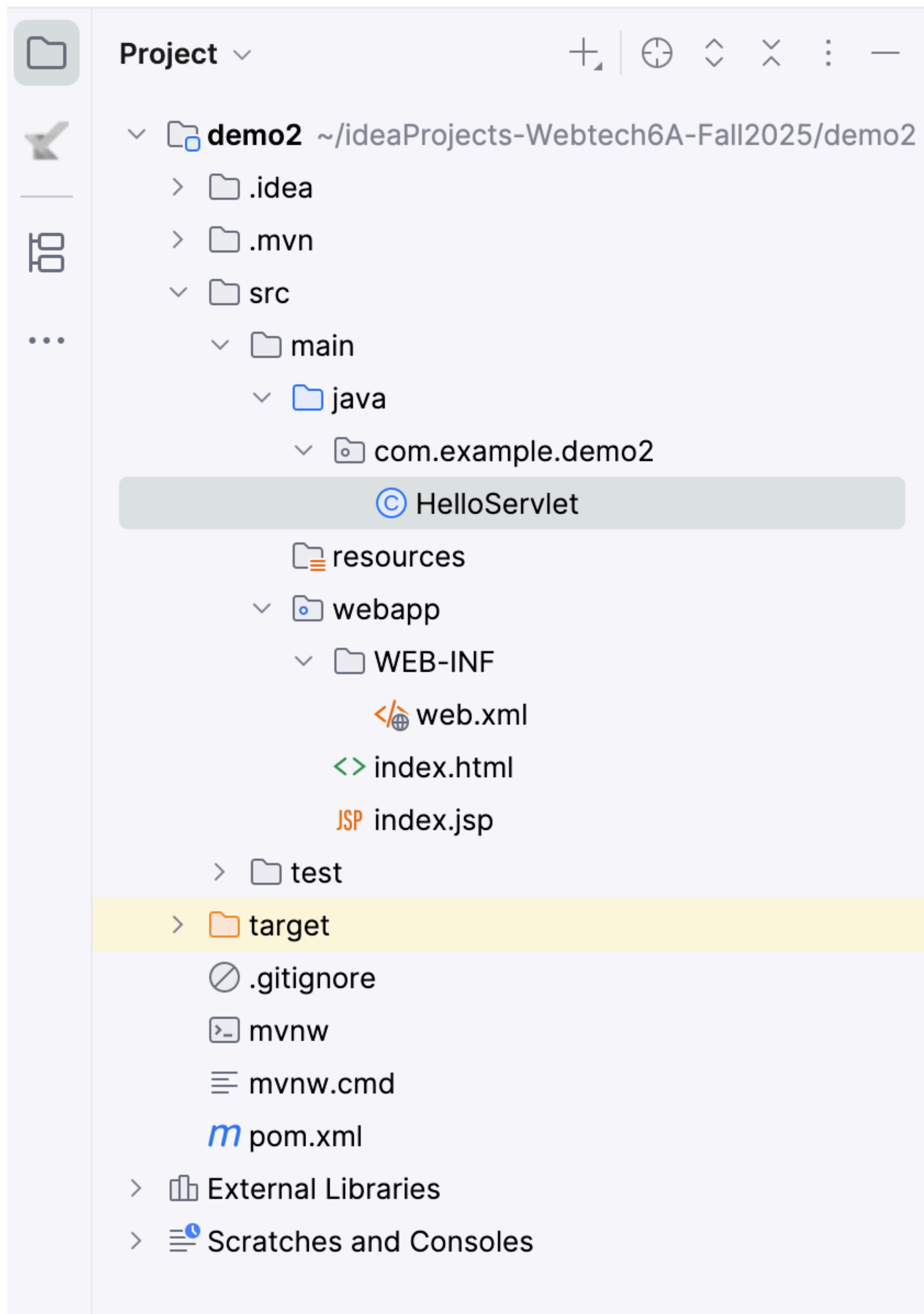
response

Apache Tomcat

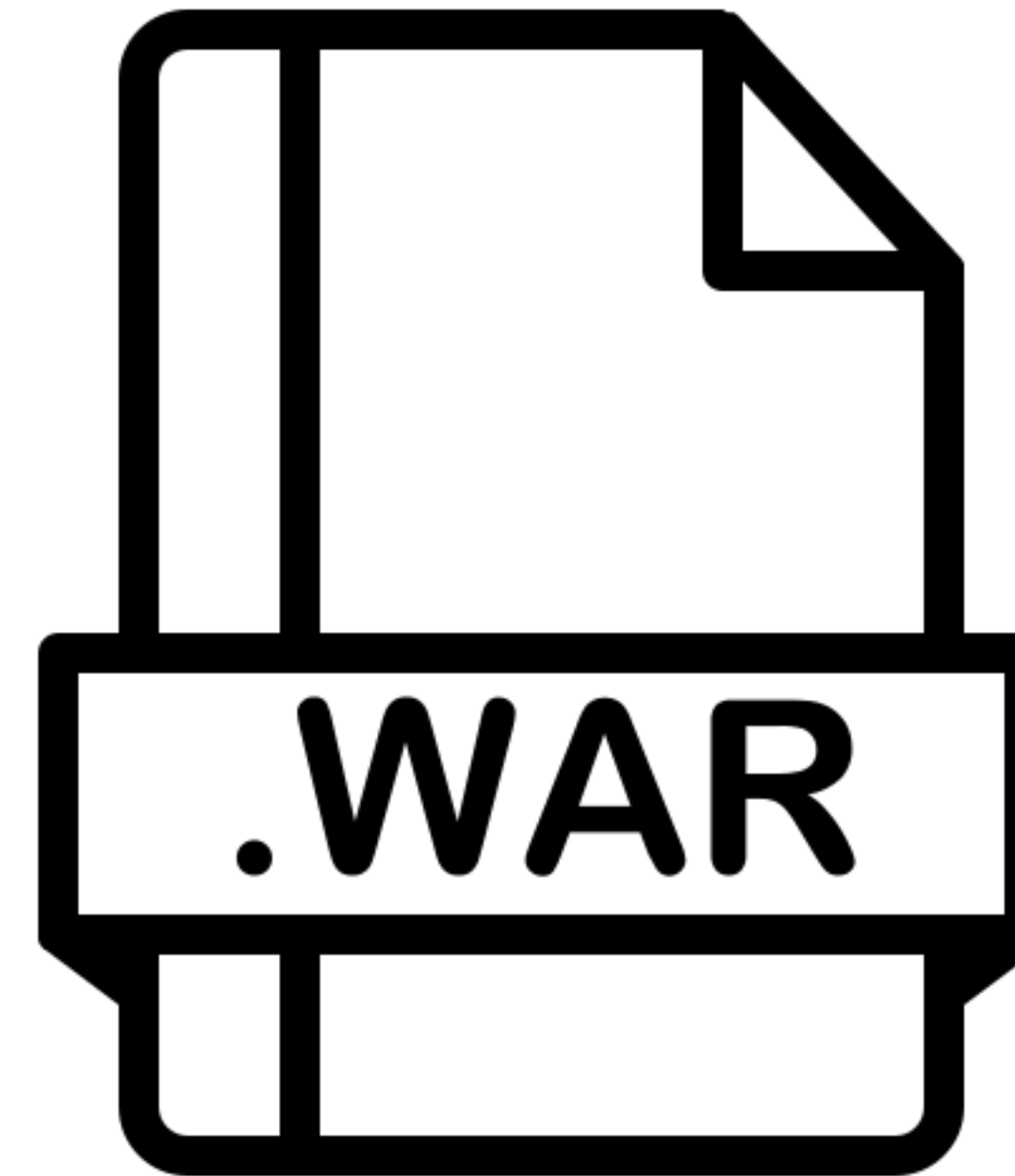
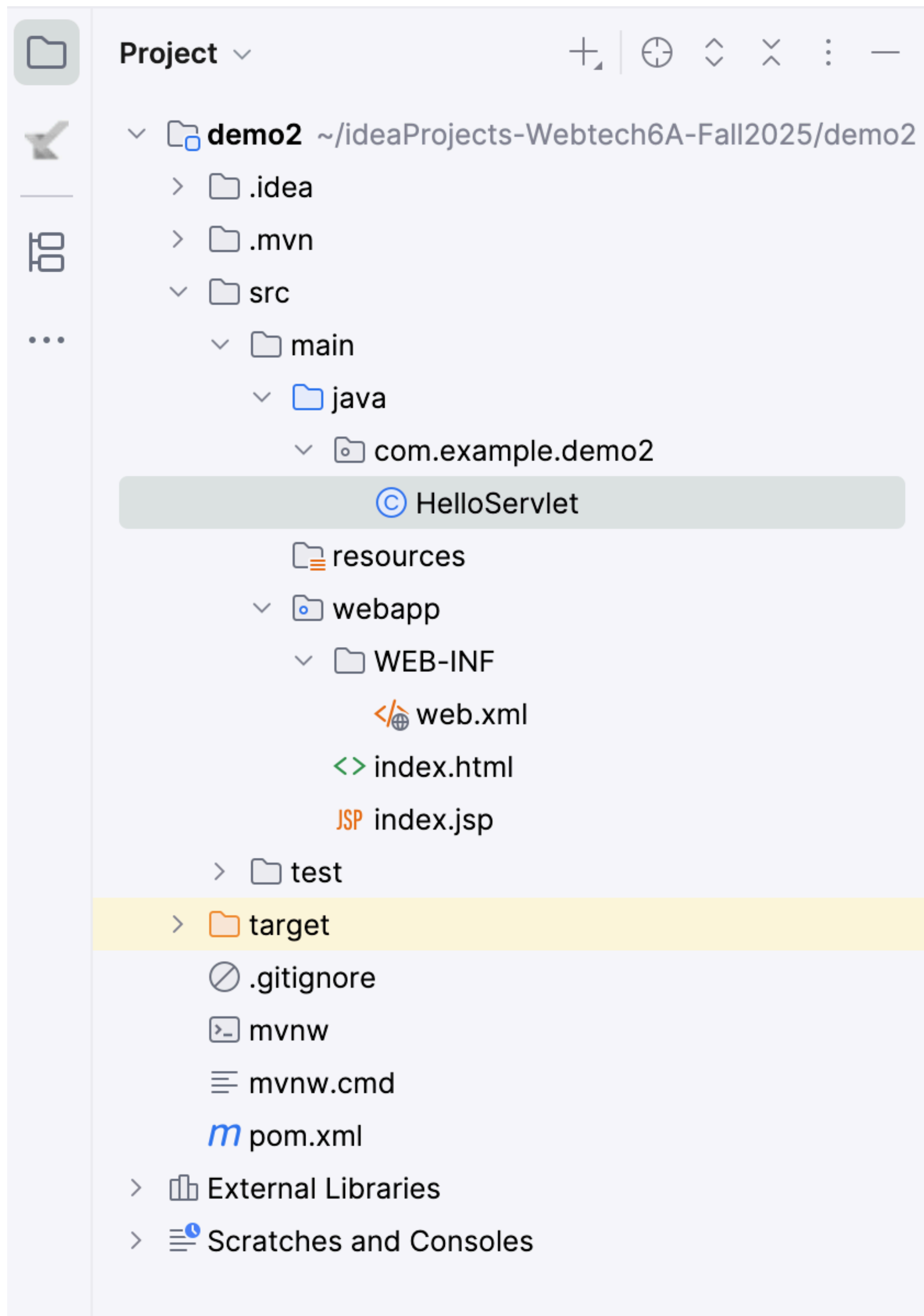
Resolve



Development View



We see a different structure at the time of development

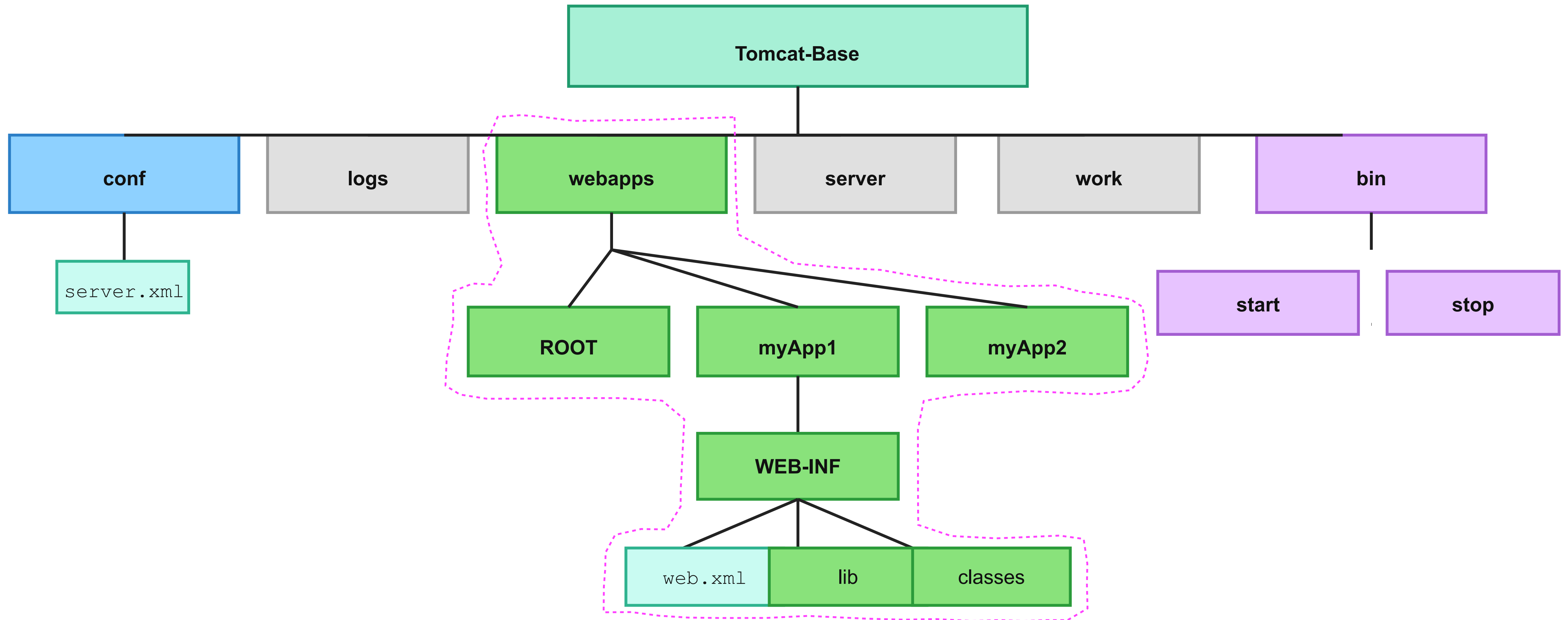


Developer focus

At the time of development, tooling show us what we need to see in order to produce a deployable artifact called a war file. The war file is your complete application that needs to be deployed on the server.

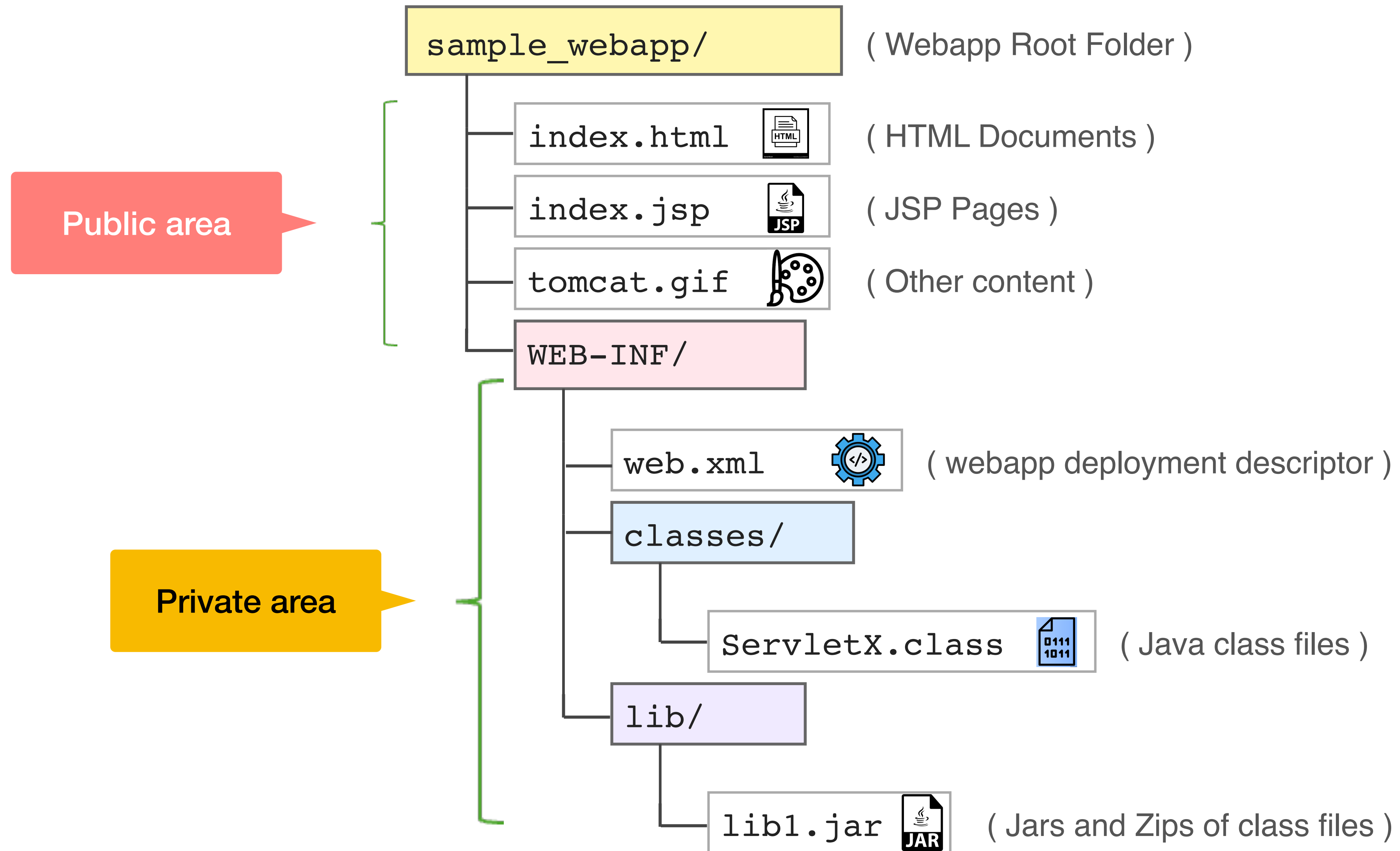
In fact its the web-apps directory on the server.

Resolve

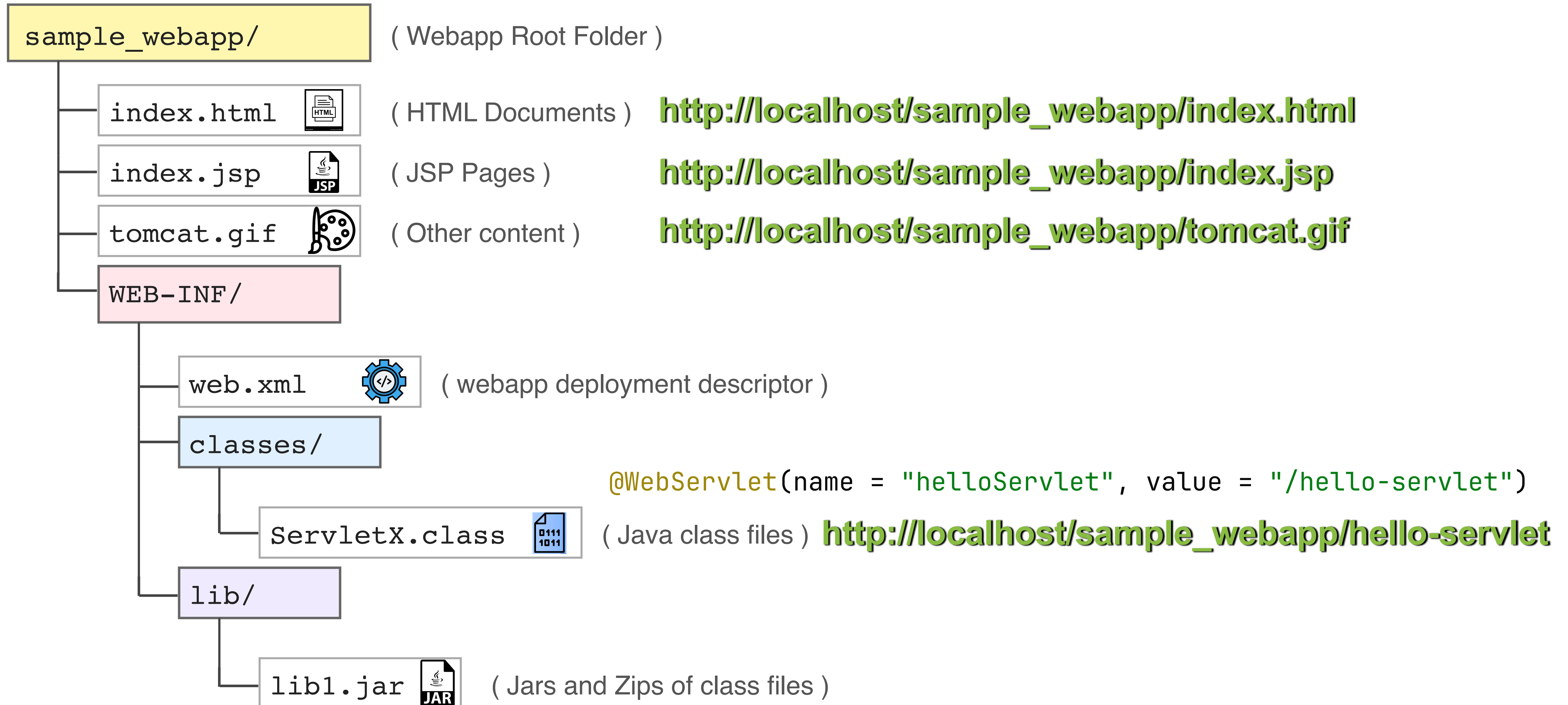


WebApp Structure

Resolve



Exercise.....Solved!



Write paths!



Servlet

I am a Java class, you can reach me with this URL

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
```

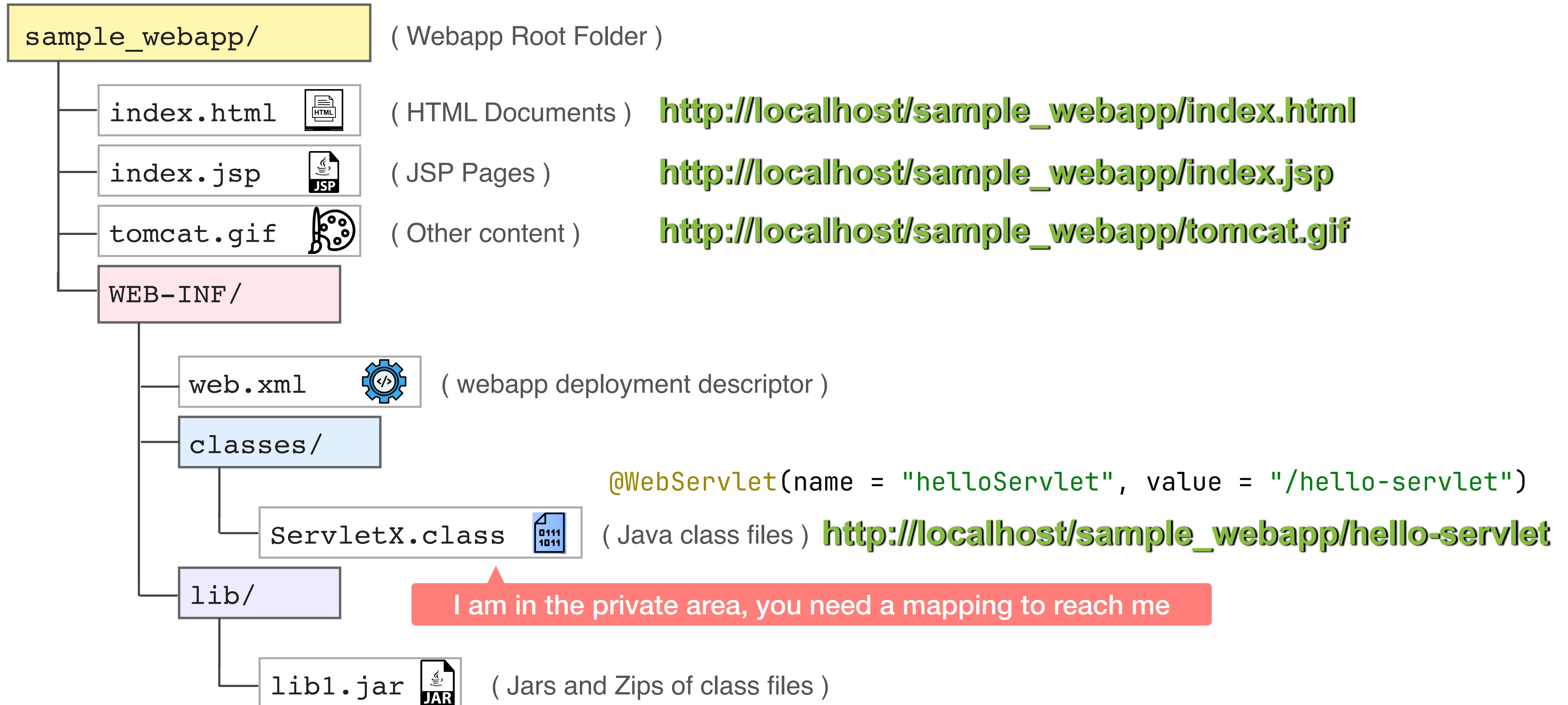
Parent class

```
public class HelloServlet extends HttpServlet {  
    @Override  
    public void doGet(HttpServletRequest request,  
        HttpServletResponse response)  
        throws IOException {  
        response.setContentType("text/html");
```

I am writing this response using sockets back to the client

```
        PrintWriter out = response.getWriter();  
        out.println("<html><body>");  
        out.println("</body></html>");  
        out.println("<h1>" + message + "</h1>");  
    }  
}
```

Servlet Mapping



Using Web.xml

Mapping: Web.xml

<web-app>

<servlet>

<servlet-name> hello</servlet-name>

<servlet-class> **HelloWorld** </servlet-class>

</servlet>

<servlet-mapping>

<servlet-name> hello</servlet-name>

<url-pattern> **/hi** </url-pattern>

</servlet-mapping>

</web-app>

Mapping: Annotations

Tag for tomcat to recognize this servlet mapping

```
@WebServlet(name = "hello", value = "/hi")
```

Mapped to url ending with /hi

```
public class HelloServlet extends HttpServlet {
```

```
}
```

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
```

Parent class

```
public class HelloServlet extends HttpServlet {
```

```
@Override
```

Re-Defined from the parent class

```
public void doGet(HttpServletRequest request,  
    HttpServletResponse response)
```

```
throws IOException {
```

```
response.setContentType("text/html");
```

```
    PrintWriter out = response.getWriter();
```

```
    out.println("<html><body>");
```

```
    out.println("</body></html>");
```

```
    out.println("<h1>Hello World</h1>");
```

```
}
```

```
}
```

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
```

Parent class

```
public class HelloServlet extends HttpServlet {
```

```
    @Override
```

Defined with two arguments request & response

```
    public void doGet(HttpServletRequest request,  
                      HttpServletResponse response)
```

```
        throws IOException {
```

```
        response.setContentType("text/html");
```

```
        PrintWriter out = response.getWriter();
```

```
        out.println("<html><body>");
```

```
        out.println("</body></html>");
```

```
        out.println("<h1>Hello World</h1>");
```

```
    }
```

```
}
```

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
```

Parent class

```
public class HelloServlet extends HttpServlet {
```

```
    @Override
```

```
    public void doGet(HttpServletRequest request,  
                      HttpServletResponse response)
```

```
        throws IOException {
```

```
        response.setContentType("text/html");
```

Request is used to receive
parameters from client

```
        PrintWriter out = response.getWriter();
```

```
        out.println("<html><body>");
```

```
        out.println("</body></html>");
```

```
        out.println("<h1>Hello World</h1>");
```

```
    }
```

```
}
```

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
```

Parent class

```
public class HelloServlet extends HttpServlet {
```

```
    @Override
```

```
    public void doGet(HttpServletRequest request,  
        HttpServletResponse response)
```

```
        throws IOException {
```

```
        response.setContentType("text/html");
```

Response is used to write some html content back to the client

```
        PrintWriter out = response.getWriter();
```

```
        out.println("<html><body>");
```

```
        out.println("</body></html>");
```

```
        out.println("<h1>Hello World</h1>");
```

```
    }
```

```
}
```

How to send parameters
to a Servlet with get
method?

QueryString

<http://www.google.co.uk/search?hl=en&q=Football>

Begins with a ? Then the first param

Succeeding all params begin with &

Receive params inside
Servlet?

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response) throws IOException {
        response.setContentType("text/html");
        // received params here
        String name = request.getParameter("name");
        String password = request.getParameter("password");
        Note you are getting params using request object's method
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("</body></html>");
        Consumed user here
        out.println("<h1>" + name + "</h1>");
        out.println("<h3> this is your password" + password + "</h3>");
    }
}
```

Consumed password here

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {
```

```
    @Override
```

```
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response) throws IOException {
```

```
        response.setContentType("text/html");
```

```
        // received params here
```

```
        String name = request.getParameter("name");
```

NullPointerException, name !=Name

```
        String password = request.getParameter("password");
```

```
        PrintWriter out = response.getWriter();
```

```
        out.println("<html><body>");
```

```
        out.println("</body></html>");
```

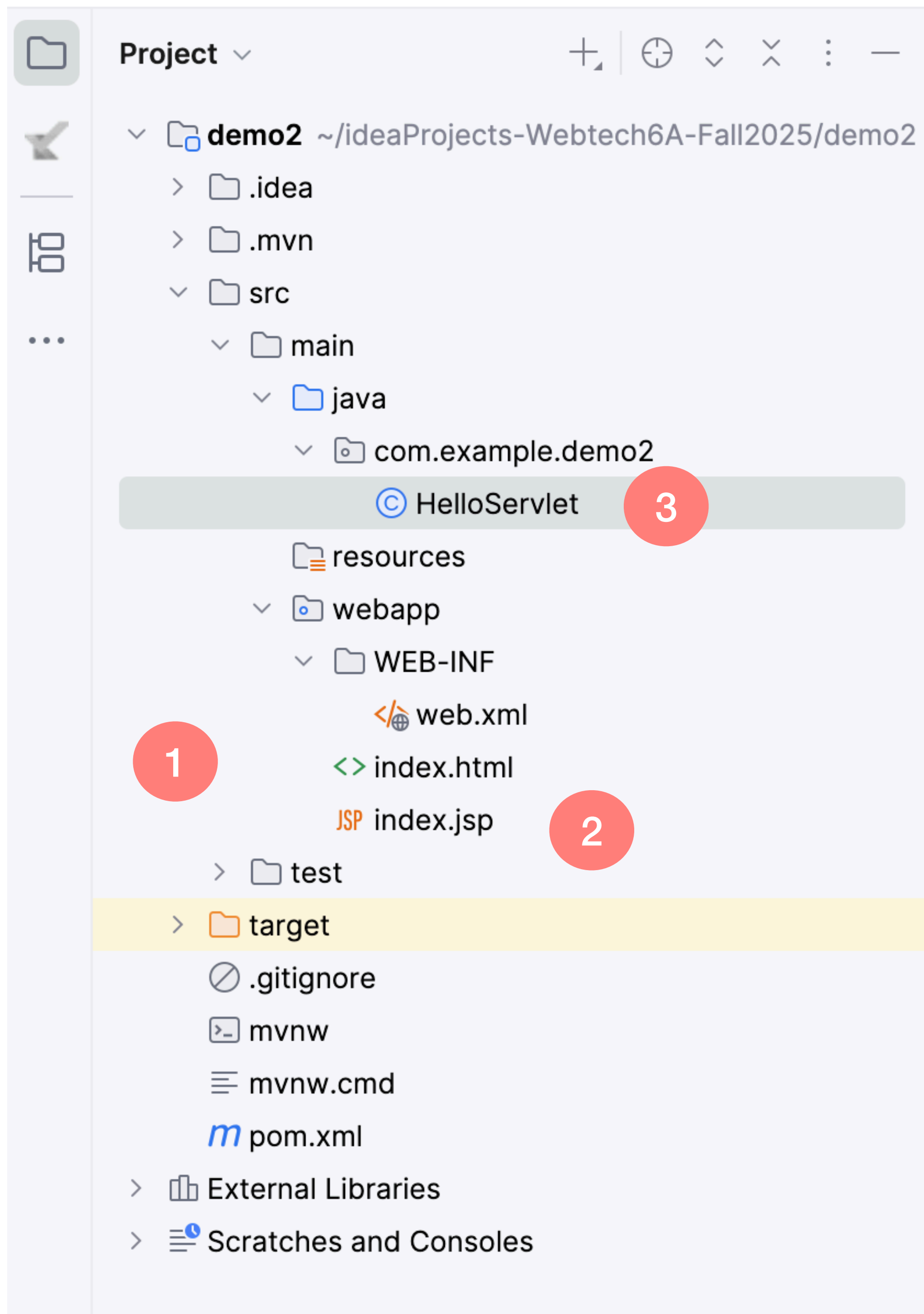
```
        out.println("<h1>" + name + "</h1>");
```

```
        out.println("<h3> this is your password" + password + "</h3>");
```

```
    }
```

<http://localhost/demo/hello-servlet?Name=ali&password=123&age=23>

Your app works like this!



2 dynamic and 1 static
resource within your
webapp!

request

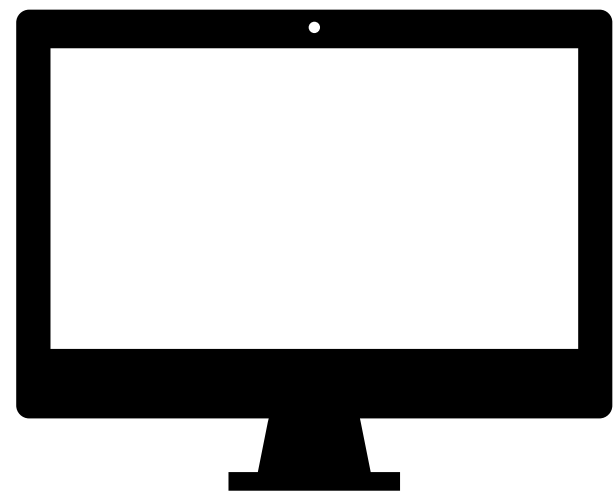
App resides within
Tomcat Server

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("</body></html>");
        out.println("<h1>" + message + "</h1>");

    }
}
```



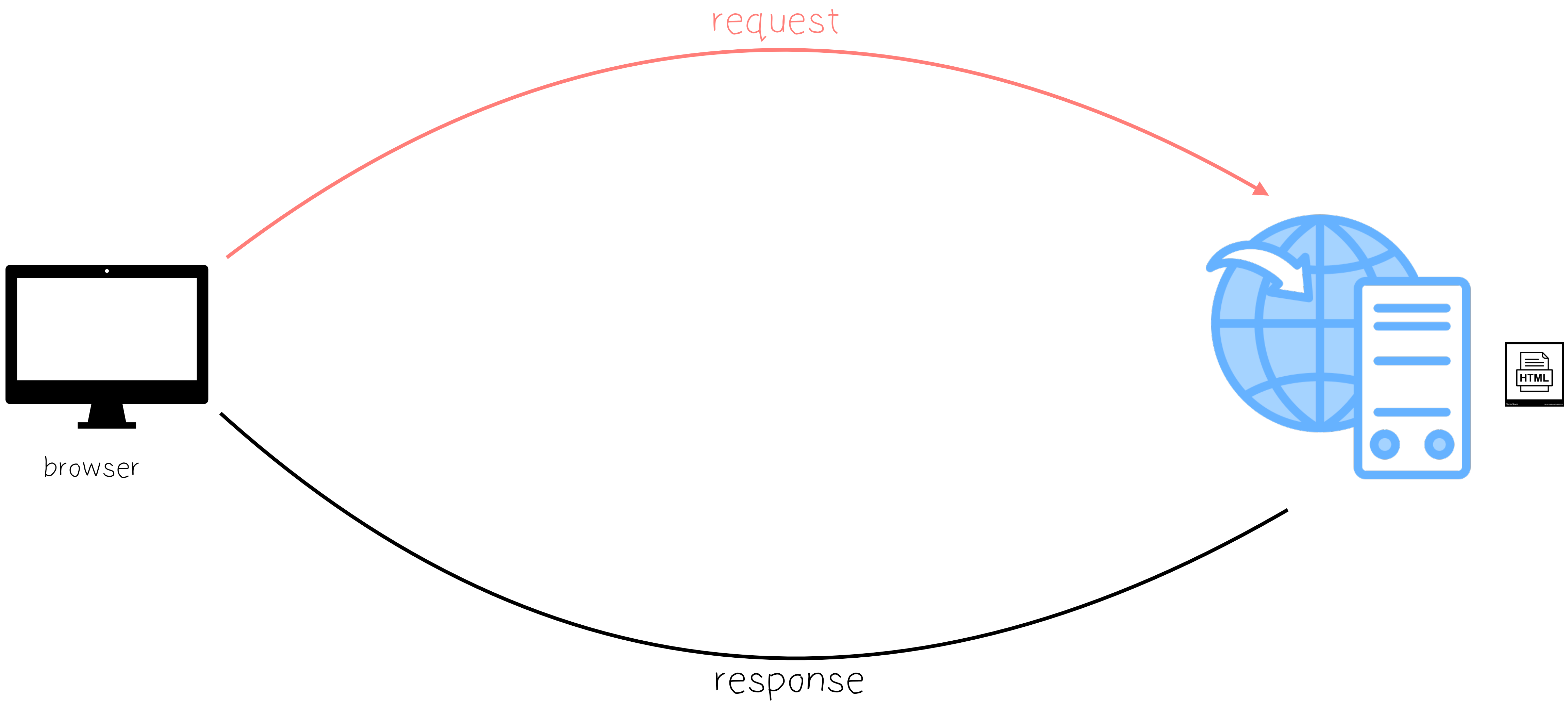
browser

response

<http://localhost/demo/hello-servlet?Name=ali&password=123&age=23>

https://

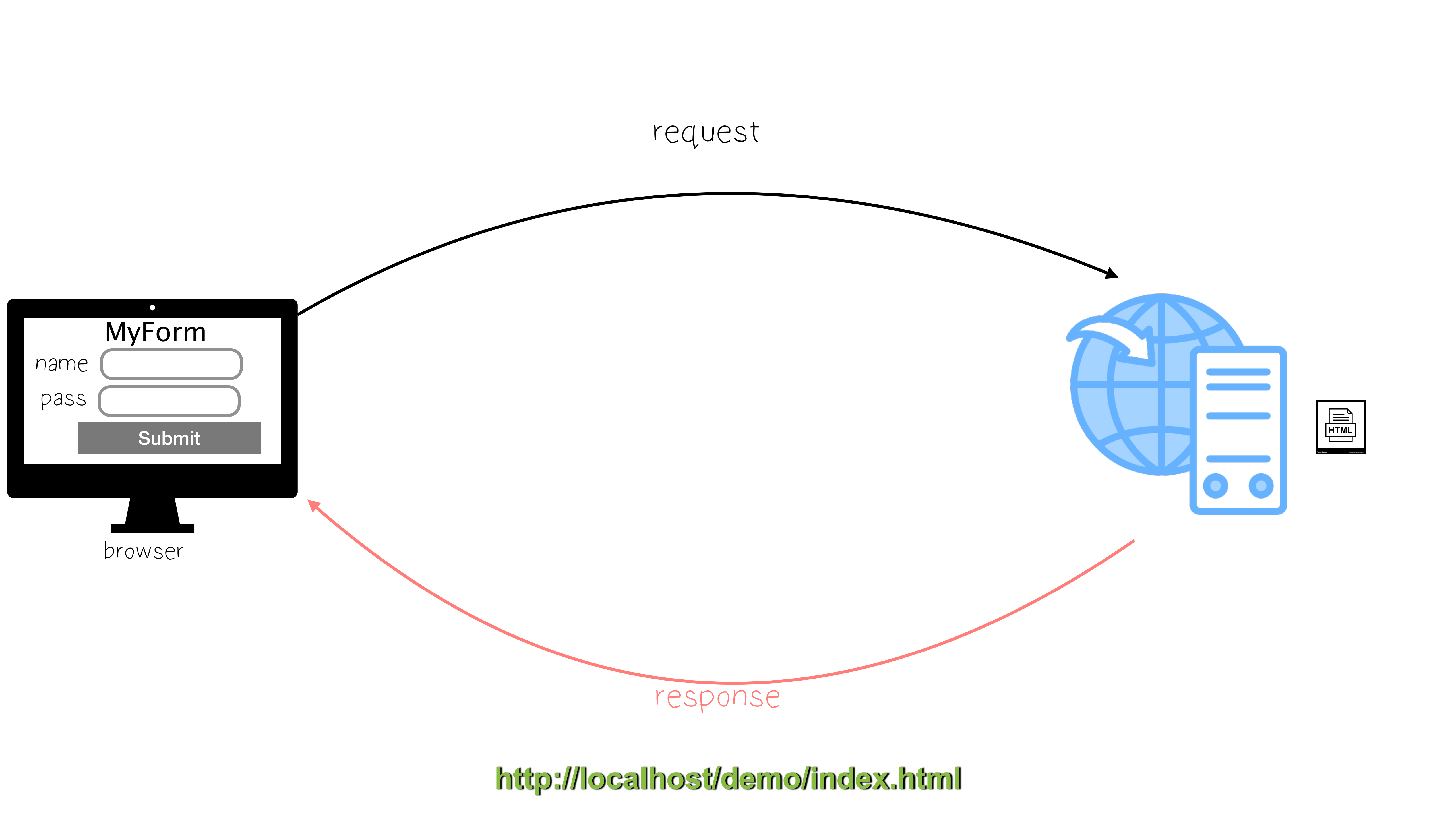




http://localhost/demo/index.html

https://

Q



```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
<h1>My form!</h1><br>
<form method="post" action="hello-servlet">
    <input type="text" name="name" placeholder="name"><br>
    <input type="password" name="password" placeholder="password"><br>
    <input type="submit">

</form>
</body>
</html>
```

Server shows nothing?

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")  
public class HelloServlet extends HttpServlet {
```

```
    @Override
```

```
    public void doGet(HttpServletRequest request,  
                      HttpServletResponse response) throws IOException {
```

```
        response.setContentType("text/html");
```

```
        // received params here
```

```
        String name = request.getParameter("name");
```

Param match with the input type name

```
        String password = request.getParameter("password");
```

```
        PrintWriter out = response.getWriter();
```

```
        out.println("<html><body>");
```

```
        out.println("</body></html>");
```

```
        out.println("<h1>" + name + "</h1>");
```

```
        out.println("<h3> this is your password" + password + "</h3>");
```

```
    }
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
<h1>My form!</h1><br>
<form method="post" action="hello-servlet">
  <input type="text" name="name" placeholder="name"><br>
  <input type="password" name="password" placeholder="password"><br>
  <input type="submit">

</form>
</body>
</html>
```

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")  
public class HelloServlet extends HttpServlet {
```

It's because of wrong http method

```
@Override  
public void doGet(HttpServletRequest request,  
                  HttpServletResponse response) throws IOException {  
    response.setContentType("text/html");  
    // received params here  
    String name =request.getParameter("name");  
    String password =request.getParameter("password");  
  
    PrintWriter out = response.getWriter();  
    out.println("<html><body>");  
    out.println("</body></html>");  
    out.println("<h1>" + name + "</h1>");  
    out.println("<h3> this is your password" + password + "</h3>");  
}
```

Code

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")  
public class HelloServlet extends HttpServlet {
```

```
@Override
```

Corrected finally!

```
    public void doPost(HttpServletRequest request,  
                        HttpServletResponse response) throws IOException {  
        response.setContentType("text/html");  
        // received params here  
        String name = request.getParameter("name");  
        String password = request.getParameter("password");  
  
        PrintWriter out = response.getWriter();  
        out.println("<html><body>");  
        out.println("</body></html>");  
        out.println("<h1>" + name + "</h1>");  
        out.println("<h3> this is your password" + password + "</h3>");  
    }
```

Assignment

Login Functionality

Use the code given in the previous slides to make a login functionality in your web-app. You have a form which captures userid/password from the user.

You are sending this to the server, server should match it with existing userid/password and send back a msg saying you are logged in.

UserDB

